INFRASTRUCTURE RUNBOOK

VC

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

VCF Depot Migration USB SATA → Internal NVMe

Procedure for relocating the offline VCF depot from a USB-attached MX500 SATA SSD to internal NVMe to eliminate the OVA-download bottleneck during VCFA and other VCF deploys.

VCF DEPOT | NVME | ROBOCOPY | HTTPS_SERVER.PY | 7920

VCF 9.0 VMware Cloud Foundation

VCF Offline Depot Migration — USB MX500 SATA → Internal NVMe

Prepared by: Virtual Control LLC **Date:** May 18, 2026 **Document Version:** 1.0 **Classification:** Internal — Lab Environment **Source:** `I:\VCF-Depot` on USB-attached Crucial MX500 SATA SSD **Destination:** `C:\VCF-Depot` on internal 4 TB NVMe (post-Macrium clone) **Workstation:** Dell Precision 7920 Tower

Scope. This document is the runbook for moving an existing VCF offline depot off a USB-attached SATA SSD to internal NVMe to remove the OVA-download bottleneck that surfaced during the 2026-05-18 VCFA Simple deploy attempt. The depot's `https_server.py` has hardcoded source paths that must be updated during the move. Includes verified commands, robocopy exit-code interpretation, and a 4-line script diff.

Companion docs. - `Fleet-vCenter-Trust-Cascade-Repair-2026-05-18.md` — the trust-state cascade that exposed this depot bottleneck - `VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md` — the deploy gotchas this migration is intended to clear up for next attempt

---

Table of Contents

---

1. Why Move the Depot

The VCF offline depot serves OVA templates (multi-GB binaries) to Fleet Management over HTTPS. Read throughput from the depot host's underlying disk is the **single largest factor in deploy duration** for any product whose binaries are not yet cached locally on Fleet.

The lab's previous depot lived at `I:\VCF-Depot` on a USB-attached Crucial MX500 SATA SSD. Read throughput from USB SATA is capped at roughly **400 MB/s** by the bus, and in practice the small-file mix of an unpacked OVA depot

delivers far less — in the 100–200 MB/s range. By comparison, internal NVMe delivers 3–7 GB/s sequential reads.

The 2026-05-18 VCFA Simple deploy stalled with the vCenter-side "Deploy OVF template" task at 0% for 2 hours 14 minutes. Fleet's parent task was still "In Progress" and `VC_DATA_COLLECTION` sub-tasks were completing periodically, but the OVA itself never finished transferring. Investigation revealed:

1. The depot's `https_server.py` Python process was alive but had stopped listening on port 8443 (apparent thread/socket exhaustion after days of uptime).
2. Restarting on USB would have recovered the deploy, but the underlying read throughput problem would recur for every subsequent OVA download.
3. With the C:\ drive now on 4 TB NVMe (Macrium clone completed 2026-05-18) and OneDrive sync paused 24 h, NVMe is the obvious target.

> **Expected outcome.** OVA pull throughput from depot to Fleet limited by network (not disk) post-migration. For Fleet (192.168.1.95) on a /24 network with the depot at 192.168.1.160, expect ~110 MB/s sustained over GigE — an order of magnitude faster than the previous USB-disk-bound rate.

---

2. Prerequisites

Item	Requirement
Free space on destination	1.2x the source size (for safety margin during partial cutover). Verified: source 287.1 GB, destination C:\ had 3,074 GB free.
`https_server.py` location	`I:\VCF-Depot\https_server.py` (copied to destination by robocopy in §3.3)
Python	C:\Python314\python.exe present (matches what was already running)
Cert + key	`server.crt` + `server.key` in depot root (copied by robocopy)
Firewall	Inbound 8443 rule already in place from the I:\ deployment — no firewall changes needed (same workstation, same port)
OneDrive sync	Should be paused or otherwise NOT including C:\VCF-Depot in synced folders
Active deploys	If a Fleet deploy is in flight and stalled because depot is down, this migration is safe; if a deploy is actively downloading, wait for it to complete before migration

---

3. Procedure

3.1 Step 1 — Stop Any Running Depot Process

If a `python.exe` process is currently serving the depot, stop it. **Carefully identify** the right PID first — the workstation may have other Python processes running (Health Check GUI, dev work, etc.).

```
 # Identify ALL python processes with command line
Get-CimInstance Win32_Process -Filter "Name='python.exe'" |
  Select-Object ProcessId, CommandLine | Format-List
```

```
# Only stop the one whose CommandLine matches https_server.py:
$pids = (Get-CimInstance Win32_Process -Filter "Name='python.exe'" |
  Where-Object { $_.CommandLine -match 'https_server\.py' }).ProcessId
foreach ($p in $pids) {
  Write-Output "Stopping depot PID $p"
  Stop-Process -Id $p -Force
}
```

Lesson from 2026-05-18 incident: the only running python.exe on the workstation was identified by a single PID number alone (34912) without inspecting its CommandLine. It turned out to be the `vcf-health-check-gui.py` app, not the depot. Stopping it had no effect on the depot (which was already down) but did kill an unrelated app. **Always filter by** `CommandLine -match 'https_server\.py'` before stopping.

3.2 Step 2 — Verify Source Size and Destination Free Space

```
$src = (Get-ChildItem 'I:\VCF-Depot' -Recurse -ErrorAction SilentlyContinue |
        Measure-Object -Property Length -Sum).Sum / 1GB
$free = (Get-Volume C).SizeRemaining / 1GB
"{0,-30} {1,8:N1} GB" -f 'Source I:\VCF-Depot',  $src
"{0,-30} {1,8:N1} GB" -f 'Destination C: free',  $free
"{0,-30} {1,8:N1} GB needed (1.2x)" -f 'Headroom check', ($src * 1.2)
```

Verified 2026-05-18: source 287.1 GB, C:\ free 3,074.7 GB.

3.3 Step 3 — Robocopy Source → Destination

```
robocopy "I:\VCF-Depot" "C:\VCF-Depot" `
  /E `
  /MT:16 `
  /R:1 /W:1 /NP `
  /LOG:"C:\Users\valca\AppData\Local\Temp\robocopy-depot.log"
```

Flag meanings:

Flag	Meaning
`/E`	Copy all subdirectories including empty ones
`/MT:16`	16 multi-threaded copy operations
`/R:1 /W:1`	Retry once with 1-second wait on transient errors
`/NP`	No per-file progress (cleaner log)
`/LOG:...`	Log destination

Expected duration on Precision 7920 with USB MX500 source: **~17 minutes for 287 GB** (verified 2026-05-18: 13:59:35 → 14:16:38, exit code 1).

Robocopy exit code 1 is SUCCESS, not failure. Robocopy uses bit-flag exit codes:

Code	Meaning
0	No files copied; source and dest already match
1	**Files copied successfully**
2	Extra files in destination (would be cleaned up with /MIR)
3	Files copied + extra files cleaned
4–7	Mismatch or non-fatal warnings
8+	Errors

Generic automation frameworks (the Claude Code background-task harness, CI shell wrappers, etc.) typically treat **any non-zero exit code as failure**. Always interpret robocopy exit codes against the table above, not against the framework's pass/fail label.

Verify size match after completion:

```
 $srcBytes = (Get-ChildItem 'I:\VCF-Depot' -Recurse -ErrorAction SilentlyContinue |
             Measure-Object -Property Length -Sum).Sum
$dstBytes = (Get-ChildItem 'C:\VCF-Depot' -Recurse -ErrorAction SilentlyContinue |
             Measure-Object -Property Length -Sum).Sum
"{0,-12} {1,12:N0} bytes" -f 'Source',      $srcBytes
"{0,-12} {1,12:N0} bytes" -f 'Destination', $dstBytes
"{0,-12} {1,12:N0} bytes" -f 'Delta',       ($srcBytes - $dstBytes)
```

Delta should be 0. Verified 2026-05-18: both 287.1 GB.

3.4 Step 4 — Edit https_server.py Paths

The script has three hardcoded `I:\\VCF-Depot` references that must be updated:

```
 # Line 15
- CERT_FILE = 'I:/VCF-Depot/server.crt'
+ CERT_FILE = 'C:/VCF-Depot/server.crt'

# Line 16
- KEY_FILE = 'I:/VCF-Depot/server.key'
+ KEY_FILE = 'C:/VCF-Depot/server.key'

# Line 83 (inside run_server)
-     os.chdir('I:/VCF-Depot')
+     os.chdir('C:/VCF-Depot')
```

Apply via PowerShell (no editor needed):

```
 $file = 'C:\VCF-Depot\https_server.py'
$content = Get-Content $file -Raw
$content = $content -replace "I:/VCF-Depot", "C:/VCF-Depot"
Set-Content $file -Value $content -NoNewline

# Verify
Select-String -Path $file -Pattern 'C:/VCF-Depot|I:/VCF-Depot' |
  Format-Table LineNumber, Line -AutoSize
```

Expected output: 3 matches, all `C:/VCF-Depot`, none `I:/VCF-Depot`.

> **Tip:** For a more portable script, change to relative paths (`server.crt` , `server.key` , `.`) and run the script with cwd = depot directory. Avoids the need to edit on future moves.

3.5 Step 5 — Start the New Depot

Run interactively in a dedicated PowerShell window (so output and Ctrl+C are accessible):

```
 cd C:\VCF-Depot
C:\Python314\python.exe https_server.py
```

Expected startup output:

```
 VCF Offline Depot Server
========================
Serving: C:\VCF-Depot
URL: https://192.168.1.160:8443/
Credentials: admin / admin
TLS: 1.2 - 1.3
Press Ctrl+C to stop
```

If the script crashes immediately:

Error	Likely cause	Fix
`FileNotFoundError: server.crt`	Cert file not copied	Re-run robocopy with `/E` (not `/MIR` — `/MIR` would have deleted it)
`OSError: [Errno 10048]` Address already in use	Old depot still bound to 8443	Find and kill the prior python (see §3.1)
`ssl.SSLError: ...PEM lib...`	Cert/key mismatch or wrong format	Verify with `openssl x509 -in server.crt -noout -dates`
`ImportError: No module named ...`	Python 3.14 mismatch with module path	Verify `C:\Python314\python.exe --version`

3.6 Step 6 — Verify and Watch for Fleet Pickup

From the same or another PowerShell session, verify the depot is reachable:

```
 # TCP-level test
Test-NetConnection -ComputerName 192.168.1.160 -Port 8443 -InformationLevel Quiet

# HTTPS-level test (workstation-local)
[System.Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
Invoke-WebRequest -Uri 'https://192.168.1.160:8443/' -UseBasicParsing -TimeoutSec 5

# HTTPS-level test from Fleet appliance (via SSH)
ssh root@192.168.1.95 "curl -sk -o /dev/null -w 'HTTP %{http_code} time=%{time_total}s\n'
https://192.168.1.160:8443/"
```

All three should return 401 (basic-auth challenge) or 200 with bytes. If you get HTTP 000 / connection refused, the depot isn't listening — check the python window for tracebacks.

If a Fleet deploy was stalled because depot was down, watch for the next `VC_DATA_COLLECTION` cycle to trigger OVA download:

```
 # On Fleet
tail -F /var/log/vrlcm/vmware_vrlcm.log | grep -iE "Downloading|Transfer|OVF|VC_DATA_COLLECTION"
```

If the deploy doesn't pick up within 15 minutes, it has likely abandoned the OVA download internally even though the parent task still shows "In Progress." Cancel via API (see `Fleet-Operations-Integration-Recovery-RCA.md` §6) and resubmit.

---

4. Rollback

```
 # Stop new depot
$pids = (Get-CimInstance Win32_Process -Filter "Name='python.exe'" |
  Where-Object { $_.CommandLine -match 'C:\\VCF-Depot\\https_server\.py' }).ProcessId
foreach ($p in $pids) { Stop-Process -Id $p -Force }

# Restart on I:\
cd I:\VCF-Depot
C:\Python314\python.exe https_server.py # original script unchanged on I:\
```

Source depot at `I:\VCF-Depot` remains intact (robocopy `/E` does not delete the source). Removing `C:\VCF-Depot` is optional; the data is duplicated until you explicitly delete it.

```
 # Optional: free up C: space after confirming I:\ works
Remove-Item -Recurse -Force C:\VCF-Depot
```

---

5. Symptoms This Fixes

You should run this migration when ALL of these apply:

Symptom	Verification command
Fleet deploys show extremely slow OVA download	vCenter Tasks pane: "Deploy OVF template" task at 0% or low % for >1 hour
Depot disk read throughput < 200 MB/s sustained	`Get-Counter '\PhysicalDisk(_total)\Disk Read Bytes/sec'` during deploy
Depot is on USB-attached SATA SSD	`Get-PhysicalDisk` shows BusType=USB for the depot disk
Internal NVMe with sufficient free space exists	`Get-Volume` shows >= 1.2x depot size free on an NVMe-backed volume

If the depot is already on internal NVMe, this migration is unnecessary — investigate other bottlenecks (Fleet's Java OVA-import buffer, network MTU, vSAN deduplication overhead, etc.).

---

6. Incident Log — 2026-05-18

Property	Value
Trigger	VCFA Simple deploy stalled at Deploy OVF template = 0% for 2h 14m
Underlying depot issue	`https_server.py` python process alive but not listening on 8443 (apparent socket exhaustion after days of uptime)
Source size	287.1 GB at `I:\VCF-Depot` (USB MX500 SSD)
Destination	`C:\VCF-Depot` on 4 TB NVMe (clone completed earlier 2026-05-18 via Macrium)
Robocopy duration	**~17 minutes** (13:59:35 → 14:16:38)
Robocopy exit code	1 (success)
Path edits applied	Lines 15, 16, 83 of `https_server.py` — all `I:/VCF-Depot` → `C:/VCF-Depot`
Mistakes avoided after	Killing python by PID alone without verifying CommandLine (see §3.1 lesson)
Outcome	Pending Fleet deploy pickup at time of writing; if no pickup within 15 min, deploy will be cancelled via API and resubmitted with verified-working values from the Precheck Gotchas runbook

---

7. Lessons Learned

1. USB SATA is the depot's single biggest bottleneck. OVA downloads from depot to Fleet over GigE are network-bound on internal NVMe, disk-bound on USB SATA. Move depot to internal NVMe whenever possible — the speed difference is roughly 10x for typical workloads.

2. Robocopy exit code 1 is success. Multiple automation frameworks misinterpret it as failure. Always cross-reference the actual exit-code table when interpreting robocopy results.

3. Identify processes by CommandLine, not just by name or PID. A workstation may have multiple `python.exe` instances running (depot, GUI apps, dev work, etc.). Filter by `CommandLine -match 'https_server\.py'` before stopping anything.

4. Long-running Python HTTP servers can stop accepting connections without dying. Symptoms: `netstat -ano | findstr :8443` shows no LISTENING; `Test-NetConnection 8443` returns false; ping still works. Cause appears to be socket / file-descriptor exhaustion in the default `http.server`. Consider a watchdog that restarts the depot if `Test-NetConnection 8443` fails twice in a row.

5. The depot URL never changes during a backing-disk move. Fleet's configured depot URL (`https://192.168.1.160:8443`) is bound to the workstation IP and port, not the disk path. No Fleet config update is needed for this migration — only the script's internal paths change.

6. The 2026-05-18 incident exposed a 2-hour pre-validation loop in Fleet that masks "downloads stalled" as "deploy in progress." Periodic `VC_DATA_COLLECTION` cycles complete successfully even when the OVA download has frozen, making "is it making progress?" hard to answer from task state alone. Watch the vCenter-side OVF Deploy task percentage and the depot's request_log.txt for the truth.

---

8. Document Information

Field	Value
Title	VCF Offline Depot Migration — USB MX500 SATA → Internal NVMe
Document ID	VC-RUNBOOK-DEPOT-MOVE-20260518
Version	1.0
Issued	May 18, 2026
Author	Virtual Control LLC
Classification	Internal — Lab Environment
Related Documents	`Fleet-vCenter-Trust-Cascade-Repair-2026-05-18.md`, `VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md`, `Fleet-Trust-Recovery-After-Cert-Rotation-KB402063-2026-05-17.md`
Distribution	Lab notebook; safe to share migration procedure externally with workstation-specific paths/IPs redacted

Revision History

Version	Date	Notes
1.0	May 18, 2026	Initial release. Documents the 2026-05-18 migration of `I:\VCF-Depot` (USB MX500 SATA, 287.1 GB) → `C:\VCF-Depot` (4 TB NVMe). Includes verified robocopy command + duration, 4-line script diff for `https_server.py`, lessons on robocopy exit-code interpretation and process-identification by CommandLine.

Document End

See also: - `Fleet-vCenter-Trust-Cascade-Repair-2026-05-18.md` — the trust-state cascade that exposed this depot bottleneck - `VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md` — deploy-time gotchas (cert SAN, IP pool count, dual drafts) - `Fleet-Trust-Recovery-After-Cert-Rotation-KB402063-2026-05-17.md` — Fleet-side cert rotation companion runbook - `Fleet-Operations-Integration-Recovery-RCA.md` — broader Fleet/vcops integration RCA (instance-limit, DB cleanup) - `reference_vcf_depot_server.md` (memory entry) — depot server location and command quick-reference